



CODE SPORT

In this month's column, let us focus our attention on some of the common compiler optimisations, and on how to write code that can be well optimised by the compiler.

In last month's column, I had posed a set of questions from C/C++, algorithms and data structures for readers to practice and prepare for interviews. I would like to thank those of you who sent in your answers to the different questions. Two of our readers, R Mohan and C K Jha, answered six of the 15 questions correctly. Congratulations to both of them! However, since there was no one who answered all the questions correctly, I would like to keep the questions open for this month as well. Please do send me your answers quickly, so that we can discuss the solutions to these questions in next month's column.

One of our student readers had written to ask me about what happens under the hood in a compiler, and how it optimises the code. The best book to learn all about compilers is 'Compiler Design' by Aho, Sethi and Ullman. Though the book seems quite daunting, with considerable focus on the front-end translation aspects of compilation, it is definitely worth a read. In this month's column, we take a quick peek at the various optimisations that are typically performed by the compiler.

What happens during compilation?

As most of you would already know, compilers typically take as input a program written in a high-level language, and generate the corresponding machine code, which can run on a specified architecture. For example,

if you have a file containing C code, and you compiled it using the open source GCC compiler on your X86 machine, the GCC (gcc.gnu.org) compiler would generate the corresponding X86 machine code, which will then run on your X86 machine. Typically, the high-level language program is lowered to the machine code by the compiler. The lowering happens in multiple steps, such as translating the high-level language code to an intermediate representation, which is then lowered to machine code. Figure 1 shows the typical organisation of a compiler.

From a layman's perspective, we can consider the compiler as consisting of two pieces, the front-end and the back-end. The front-end is language-dependent, and is different for each high-level language like C, C++ or Fortran. Front-ends take source files in a high-level language and create intermediate code.

The back-end of the compiler typically consists of an optimiser, which optimises the intermediate code; and the code generator, which generates the machine code. So it is possible to have different front-ends sharing a common back-end (which is done in GCC). The back-end needs to be different for each of the different computer architectures for which it generates machine code. For instance, when compiling C programs in GCC on the x86 platform and on the Sun SPARC platform, both compilers will use the same front-end (C), but